

do expressions

Document #: P2806R1
Date: 2023-03-12
Project: Programming Language C++
Audience: EWG
Reply-to: Bruno Cardoso Lopes
<bruno.cardoso@gmail.com>
Zach Laine
<whatwasthataddress@gmail.com>
Michael Park
<mcypark@gmail.com>
Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[3 do expressions](#)

[3.1 Scope](#)

[3.2 do return statement](#)

[3.3 Type and Value Category](#)

[3.4 Copy Elision](#)

[3.5 Control Flow](#)

[3.5.1 noreturn functions](#)

[3.5.2 Always-escaping expressions](#)

[3.5.3 goto](#)

[3.5.4 Should falling off the end be undefined behavior?](#)

[3.6 Grammar Disambiguation](#)

[3.7 Prior Art](#)

[3.8 What About Reflection?](#)

[3.9 Where can do expressions appear](#)

[4 Wording](#)

[5 References](#)

§ 1 Revision History

Since [[P2806R0](#)], some more discussion about implicit last value vs explicit return, reflection, and a grammar fix to the still-incomplete wording.

§ 2 Introduction

C++ is a language built on statements. `if` is not an expression, loops aren't expressions, statements aren't expressions (except maybe in the specific case of *expression*;;).

When a single expression is insufficient, the only solution C++ currently has at its disposal is to invoke a function - where that function can now contain arbitrarily many statements. Since C++11, that function can be expressed more conveniently in the form of an immediately invoked lambda.

However, this approach leaves a lot to be desired. An immediately invoked lambda introduced an extra function scope, which makes control flow much more challenging - it becomes impossible to `break` or `continue` out of a loop, and attempting to `return` from the enclosing function or `co_await`, `co_yield`, or `co_return` from the enclosing coroutine becomes an exercise in cleverness.

You also have to deal with the issue that the difference between initializing a variable used an immediately-invoked lambda and initializing a variable from a lambda only differs in the trailing `()`, arbitrarily deep into an expression, which are easy to forget. Some people actually use `std::invoke` in this context, specifically to make it clearer that this lambda is, indeed, intended to be immediately invoked.

This problem surfaces especially brightly in the context of pattern matching [\[P1371R3\]](#), where the current design is built upon a sequence of:

```
pattern => expression;
```

This syntax only allows for a single *expression*, which means that pattern matching has to figure out how to deal with the situation where the user wants to write more than, well, a single expression. The current design is to allow `{ statement }` to be evaluated as an expression of type `void`. This is a hack, which is kind of weird (since such a thing is not actually an expression of type `void`), but also limits the ability for pattern matching to support another kind of useful syntax: `=> braced-init-list`:

```
auto f() -> std::pair<int, int> {  
    // this is fine  
    return {1, 2};  
  
    // this is ill-formed in P1371  
    return true match -> std::pair<int, int> {  
        _ => {1, 2}  
    };  
}
```

There's no way to make that work, because `{` starts a statement. So the choice in that paper lacks orthogonality: we have a hack to support multiple expressions (which are very important to support) that is inventing such support on the fly, in a novel way that is very narrow (only supports `void`), that throws other useful syntax under the bus.

What pattern matching really needs here is a statement-expression syntax. But it's not just pattern matching that has a strong desire for statement-expressions, this would be a broadly useful facility, so we

should have an orthogonal language feature that supports statement-expressions in a way that would allow pattern matching to simplify its grammar to:

```
pattern => expr-or-braced-init-list;
```

§ 3 do expressions

Our proposal is the addition of a new kind of expression, called a `do` expression.

In its simplest form:

```
int x = do { do return 42; };
```

A `do` expression consists of a sequence of statements, but is still, itself, an expression (and thus has a value and a type). There are a lot of interesting rules that we need to discuss about how those statements behave.

§ 3.1 Scope

A `do` expression does introduce a new block scope - as the braces might suggest. But it does *not* introduce a new function scope. There is no new stack frame. Which is what allows external control flow to work (see below).

§ 3.2 do return statement

The new `do return` statement has the same form as the `return` statement we have today: `do return expr-or-braced-init-listopt;`. It's behavior corresponds closely to that `return`, in unsurprising ways - `do return` yields from a `do` expression in the same way that `return` returns from a function.

While `do return value;` and `return value;` do look quite close together and mean fairly different things, the leading `do` we think should be sufficiently clear, and we think it is a good spelling for this statement.

Other alternative spellings we've considered:

- `do_return`
- `do_yield` (presented to EWG in Issaquah as the initial pre-publication draft of this proposal)
- `do yield`
- `do break` (similarly to `return`, we are breaking out of this expression, but is less likely to conflict since `break` is less likely to be used than `return` and also the corresponding `break value;` is invalid today)
- `=>` (or some other arrow, like `<-` or `<=`)

§ 3.3 Type and Value Category

The expression `do { do return 42; }` is a prvalue of type `int`. We deduce the type from all of the `do return` statements, in the same way that `auto` return type deduction works for functions and lambdas.

An explicit *trailing-return-type* can be provided to override this:

```
do -> long { do return 42; }
```

If no `do return` statement appears in the body of the `do` expression, or every `do return` statement is of the form `do return;`, then the expression is a prvalue of type `void`.

Falling off the end of a `do` expression behaves like an implicit `do return;` - if this is incompatible the type of the `do` expression, the expression is ill-formed. This is the one key difference with functions: this case is not undefined behavior. This will be discussed in more detail later.

This makes the pattern matching cases [P2688R0] work pretty naturally:

P2688R0	Proposed
<pre>x match { 0 => { cout << "got zero"; }; 1 => { cout << "got one"; }; _ => { cout << "don't care"; }; }</pre>	<pre>x match { 0 => do { cout << "got zero"; }; 1 => do { cout << "got one"; }; _ => do { cout << "don't care"; }; }</pre>

Here, the whole match expression has type `void` because each arm has type `void` because none of the `do` expressions have a `do return` statement.

Yes, this requires an extra `do` for each arm, but it means we have a language that's much easier to explain because it's consistent - `do { cout << "don't care"; }` is a `void` expression in *any* context. We don't have a *compound-statement* that happens to be a `void` expression just in this one spot.

P2688R0	Proposed
<pre>auto f(int i) { return i match -> std::pair<int, int> { 0 => {1, 2}; // ill- formed _ => std::pair{3, 4}; // ok } }</pre>	<pre>auto f(int i) { return i match -> std::pair<int, int> { 0 => {1, 2}; // ok _ => std::pair{3, 4}; // ok } }</pre>

Here, the existing pattern matching cannot support a *braced-init-list* because `{` is used for the special `void-statement-case`. But if we had `do` expressions, the grammar of pattern matching can use *expr-or-braced-init-list* in the same way that we already do in many other places in the C++ grammar. This example just works.

§ 3.4 Copy Elision

All the rules for initializing from `do` expression, and the way the expression that appears in a `do return` statement is treated, are the same as what the rules are for `return`.

Implicit move applies, for variables declared within the body of the `do` expression. In the following example, `r` is an unparenthesized *id-expression* that names an automatic storage variable declared within the statement, so it's implicitly moved:

```
std::string s = do {  
    std::string r = "hello";  
    r += "world";  
    do return r;  
};
```

Note that automatic storage variables declared within the function that the `do` expression appears, but not declared within the statement-expression itself, are *not* implicitly moved (since they can be used later).

§ 3.5 Control Flow

In a regular function, there are four ways to escape the function scope:

1. a `return` statement
2. `throwing` an exception
3. invoking a `[[noreturn]]` function, `std::abort()` and `std::unreachable()`
4. falling off the end of the function (undefined behavior if the return type is not `void`)

The same is true for coroutines, except substituting `return` for `co_return` (and likewise falling off the end is undefined behavior if there is no `return_void()` function on the promise type).

For a `do` expression, we have two different directions where we can escape (in a non-exception, non-`[[noreturn]]` case): we either yield an expression, or we escape the *outer* scope. That is, we can also:

1. `return` from the enclosing function (or `co_return` from the enclosing coroutine)
2. `break` or `continue` from the innermost enclosing loop (if any, ill-formed otherwise)

Additionally, for point (4) while we could simply (for consistency) propagate the same rules for falling-off-the-end as functions, then lambdas (C++11), then coroutines (C++20), we would like to consider not introducing another case for undefined behavior here and enforcing that the user provides more information themselves.

That is, the rule we propose that the implementation form a control flow graph of the `do` expression and consider each one of the six escaping kinds described above. All `do return` statements (including the implicit `do return`; introduced by falling off the end, if the implementation cannot prove that it does not happen) need to either have the same type (if no *trailing-return-type*) or be compatible with the provided return type (if provided). Anything else is ill-formed.

Let's go through some examples.

Example	Discussion
<pre>auto a = do { if (cond) { do return 1; } else { do return 2; } };</pre>	OK: All yielding control paths have the same type. There's no falling off the end.
<pre>auto b = do { if (cond) { do return 1; } else { do return 2.0; } };</pre>	Error: The yielding control paths have different types and there is no provided <i>trailing-return-type</i> . This would be okay if it were <code>do -> int { ... }</code> or <code>do -> double { ... }</code> or <code>do -> float { ... }</code> , etc.
<pre>auto c = do { if (cond) { do return 1; } do return 2; };</pre>	OK: Similar to a, all yielding control paths yield the same type. There is no falling off the end here, it is not important that a yielding <code>if</code> has an <code>else</code> .
<pre>auto d = do { if (cond) { do return 1; } };</pre>	Error: There are two yielding control paths here: the <code>do return 1;</code> and the implicit <code>do return;</code> from falling off the end, those types are incompatible. The equivalent in functions and coroutines would be undefined behavior in if <code>cond</code> is <code>false</code> .
<pre>int e = do { if (cond) { do return; } }, 1;</pre>	OK: As above, there are two yielding control paths here, but both the explicit and the implicit ones are <code>do return;</code> which are compatible.
<pre>int f = do { if (cond) { do return 1; } throw 2; };</pre>	OK: We no longer fall off the end here, since we always escape. There is only one yielding path.
<pre>int outer() { int g = do { if (cond) {</pre>	OK: Similar to the above, it's just that we're escaping by returning from the outer function instead of throwing. Still not falling off the end.

<pre> do return 1; } return 3; }; </pre>	
<pre> int h = do { if (cond) { do return 1; } std::abort(); }; </pre>	OK: <code>std::abort()</code> means that we cannot fall off the end, see discussion on <code>[[noreturn]]</code> below.
<pre> enum Color { Red, Green, Blue }; void func(Color c) { std::string_view name = do { switch (c) { case Red: do return "Red"sv; case Green: do return "Green"sv; case Blue: do return "Blue"sv; } }; } </pre>	Error: This is probably the most interesting case when it comes to falling off the end. Here, the user knows that <code>c</code> only has three values, but the implementation does not, so it could still fall off the end. gcc does warn on the equivalent function form of this, clang does not. The typical solution here might be to add <code>__builtin_unreachable()</code>, now <code>std::unreachable()</code>, to the end of the function, but for this to work we have to discuss <code>[[noreturn]]</code> below. Barring that, the user would have to add either some default value or some other kind of control flow (like an exception, etc).
<pre> void func() { for (;;) { int j = do { if (cond) { break; } for (something) { if (cond) { do return 1; } } do return 2; }; } </pre>	OK: The first <code>break</code> escapes the <code>do</code> expression and breaks from the outer loop. Otherwise, we have two yielding statements which both yield <code>int</code>. If the <code>do return 2;</code> statement did not exist, this would be ill-formed unless the compiler could prove that the loop itself did not terminate. If the loop were <code>for (;;)</code>, then the lack of <code>do return 2;</code> would be fine - but anything more complicated than that would require some kind of final yield (or <code>throw</code>, etc.)

```
}  
}
```

To reiterate: the implementation produces a control flow graph of the `do` expression and considers all yielding statements (*including* the implicit `do return`; on falling off the end, if the implementation considers that to be a possible path) in order to determine correctness of the statement-expression. The kinds of control flow that escape the statement entirely (exceptions, `return`, `break`, `continue`, `co_return`) do not need to be considered for purposes of consistency of yields (since they do not yield values).

§ 3.5.1 noreturn functions

The language currently has several kinds of escaping control flow that it recognizes. As mentioned, exceptions, `return`, `continue`, `break`, and `co_return`. And, allegedly, `goto`.

But there's one kind of escaping control flow that it *does not* currently recognize: functions marked `[[noreturn]]`. A call to `std::abort()` or `std::terminate()` or `std::unreachable()` escapes control flow, for sure, but today this is just an attribute:

```
int i = do {  
    if (cond) {  
        do return 5;  
    }  
  
    std::abort();  
  
    // we know control flow never gets here, so we should not need to  
    // insert an implicit "do return;"  
};
```

Pattern Matching has this same problem - it needs to support arms that might `std::terminate()` or are `std::unreachable()`, so it that proposal currently is introducing a dedicated syntax to mark an arm as non-returning: `!{ std::terminate(); }`. Which is... less than ideal.

However, the rule in 9.12.10 [\[dcl.attr.noreturn\]](#)/2 is:

- 2 If a function `f` is called where `f` was previously declared with the `noreturn` attribute and `f` eventually returns, the behavior is undefined.

That is normative wording which we can rely on. The above `do` expression can only fall off the end if `std::abort` returns, which is *already* undefined behavior. We can avoid introducing any new undefined behavior ourselves as part of this feature.

That is: invoking a function marked `[[noreturn]]` can be considering an escaping control flow in exactly the same way that `return`, `break`, `throw`, etc., are already.

§ 3.5.2 Always-escaping expressions

Consider:


```
int i = do -> int {
    throw 42;
};
```

This is weird, but might end up as a result of template instantiation where maybe other control paths (guarded with an `if constexpr`) actually had `do return` statements in them. So it needs to be allowed.

§ 3.5.3 goto

Using `goto` in a `do` expression has some unique problems.

Jumping *within* a `do` expression should follow whatever restrictions we already have (see 8.8 [\[stmt.dcl\]](#)). Jumping *into* a `do` expression should be completely disallowed (we would call the *statement* of a `do` expression a control-flow limited statement).

Jumping *out* of a `do` expression is potentially useful though, in the same way that `break`, `continue`, and `return` are:

```
for (loop1) {
    for (loop2) {
        int i = do {
            if (cond) {
                goto done;
            }

            do return value;
        };
    }
}
done:
```

Breaking out of multiple loops is one of the uses of `goto` that has no real substitute today. The above example should be fine. But referring to any label that is in scope of the variable we're initializing needs to be disallowed - since we wouldn't have actually initialized the variable. We need to ensure that the 8.8 [\[stmt.dcl\]](#) rule is extended to cover this case.

Also, while computed goto is not a standard C++ feature, it would be nice to disallow this example, courtesy of (of course) JF Bastien (in this case, we are referring to a label that is within `v`'s scope. We're not jumping to it directly, but the ability to jump to it indirectly is still problematic):

```
#include <stdio.h>

struct label {
    static inline void* e;
    int v;

    label()
    try
        : v(({
            fprintf(stderr, "oh\n");
            e = &&awesome;
            throw 1;
        })
```

```

        42;
    })))
{
    fprintf(stderr, "no\n");
    awesome:
    fprintf(stderr, "you\n");
} catch(...) {
    fprintf(stderr, "don't\n");
    goto *e;
}
};

int main() {
    label l;
}

```

§ 3.5.4 Should falling off the end be undefined behavior?

Consider:

```

int i = do {
    if (cond) {
        do return 0;
    }
};

```

Is this statement ill-formed (because there is a control path that falls off the end of the `do` expression, as discussed in this section) or should this statement be undefined behavior? The latter would be consistent with functions, lambdas, and coroutines (and not a if-you-squint-enough kind of consistency either, this would be exactly identical).

It would make for a simpler design if we adopted undefined behavior here, but we think it's a better design to force the user to cover all control paths themselves.

§ 3.6 Grammar Disambiguation

We have to disambiguate between a `do` expression and a `do-while` loop.

In an expression-only context, the latter isn't possible, so we're fine there.

In a statement context, a `do` expression is completely pointless - you can just write statements. So we disambiguate in favor of the `do-while` loop. If somebody really, for some reason, wants to write a `do` expression statement, they can parenthesize it: `(do { do return 42; })`. A statement that begins with a `(` has to then be an expression, so we're now in an expression-only context.

We also have to disambiguate between a `do return` statement (if that is the chosen spelling) and a `do-while` loop whose *statement* is a `return` statement:

```

do return value; while (cond);

```

This could be parsed as a `do return` statement followed by an infinite loop (that would never be executed because we've already returned out of the expression) or as a `do-while` loop containing a single, unbraced, return statement.

The latter interpretation is valid code today, but is completely useless as it is exactly equivalent to having written `return value;` to begin with, so we think it's reasonable to disambiguate in favor of the former interpretation. This isn't a silent change in meaning, since all such code would become ill-formed by way of not appearing in a `do` expression - and the new meaning would almost surely lead to a compiler warning due to the unreachable code.

Also because we would unconditionally parse a statement as beginning with `do` as a `do-while` loop, code like this would not work:

```
do { do return X{}; }.foo();
```

Such code would also have to be parenthesized to disambiguate, which doesn't seem like a huge burden on the user.

§ 3.7 Prior Art

GCC has had an extension called [statement-expressions](#) for decades, which look very similar to what we're proposing here:

gcc	Proposed
<pre>({ int y = foo(); int z; if (y > 0) z = y; else z = -y; z; })</pre>	<pre>do { int y = foo(); if (y > 0) { do return y; } else { do return -y; } }</pre>

The reason we're not simply proposing to standardize the existing extension is that there are two features we see that are lacking in it that are not easy to add:

1. The ability to specify a return type, which is critical for allowing statement-expressions to be lvalues.
2. The ability to support yielding out of different branches of `if`, due the implicit nature of the yield.

For (1), there is simply no obvious place to put the *trailing-return-type*. For (2), you can't turn `ifs` into expressions in any meaningful way. It is fairly straightforward to answer both questions for our proposed form.

Let's also take the example motivating case from [\[P2561R1\]](#) and compare implicit last expression to explicit return:

Implicit Last Value	Explicit Return
<pre> auto foo(int i) -> std::expected<int, E> auto bar(int i) -> std::expected<int, E> { int j = do { auto r = foo(i); if (not r) { return std::unexpected(r.error()); } *r // <== NB: no semicolon }; return j * j; } </pre>	<pre> auto foo(int i) -> std::expected<int, E> auto bar(int i) -> std::expected<int, E> { int j = do { auto r = foo(i); if (not r) { return std::unexpected(r.error()); } do return *r; }; return j * j; } </pre>

In the simple cases, explicit last value (on the left) will be shorter than an explicit return (on the right). But implicit last value is more limited. We cannot do early return (by design), which means that a `do` expression would not be able to return from a loop either. We would have to extend the language to support `if` expressions, so that at the very least the first example above could be made easier - which would add more complexity to the design.

There's also the question of `void` expressions - which are where many of the pattern matching examples come from. In Rust, for instance, there is a differentiation based on the presence of a semicolon:

```

let a: i32 = { 1; 2 };
let b: () = { 1; 2; };

```

This is a simple (if silly) example of a block expression in Rust. The value of the block is the value of the last expression of the block (Rust has both `if` expressions and loop expressions) - in the first case the last example is `2`, so `a` is an `i32`, while in the second example `2;` is a statement, so the last value is the... nothing... after the `;`, which is `()` (Rust's unit type). This seems like too subtle a distinction, and one that's very easy to get wrong (although typically the types are far enough apart such that if you get it wrong it's a compiler error, rather than a runtime one):

```

auto a = do { 1; 2 }; // ok, a is an int
auto b = do { 1; 2; }; // ill-formed, b would be void (unless Regular Void
                       is adopted)

```

But this would mean that our original example would work, just for a very different reason (rather than being `void` expressions due to the lack of `do return`, they become `void` expressions due to not having a final expression):

```

x match {
    0 => do { cout << "got zero"; };
    1 => do { cout << "got one"; };
    _ => do { cout << "don't care"; };
}

```

Ultimately, we feel that the simplicity of the proposed design and its consistency and uniformity with other parts of the language outweigh the added verbosity in the simple (though typical) cases.

§ 3.8 What About Reflection?

A question that often comes up, for any language feature: if we had reflection and, in particular, code injection: would we need this facility?

The answer is not only yes, but reflection is a good motivating use-case for this facility. Because the language does not have any kind of block expression today, adding support for one would increase the amount of ways that code injection could work.

One example might be, again, the error propagation proposal in [\[P2561R1\]](#). If reflection allows me to write a hygienic macro that does code injection, perhaps we could write a library such that `try_(E)` would inject an expression that would evaluate in the way that that paper proposes. But in order to do such a thing, we would need to be able to have a block expression to inject. This paper provides such a block expression.

§ 3.9 Where can `do` expressions appear

gcc's statement-expressions are not usable in all expression contexts. Trying to use them at namespace-scope, or in a default member initializer, etc, fails:

```
int i = ({           // error: statement-expressions are not allowed outside
           functions
    int j = 2; //      nor in template-argument lists
    j;
});
```

In such contexts, there is a much smaller difference than a statement-expression and an immediately invoked lambda since you don't have any other interesting control flow that you can do - the expression either yields a value or the program terminates.

But if we're going to add a new language feature, it seems better to allow it to be used in all expression contexts - we would just have to say what happens in this case. Especially since if we're adding a feature to subsume immediately invoked lambdas, it would be preferable to subsume *all* immediately invoked lambdas, not just some or most.

We can think of a `do` expression as simply behaving like an immediately invoked lambda in such contexts. Not in the sense of allowing `return` statements (there's still no enclosing function to return out of), but the sense that any local variables declared would exist in a function stack. But this is probably more of a compiler implementation detail rather than a language design detail.

In short: `do` expressions should be usable in any expression context.

§ 4 Wording

This wording is quite incomplete, but is intended at this point to simply be a sketch to help understand the contour of the proposal.

Add to 7.5 [\[expr.prim\]](#):

```
primary-expression:  
  literal  
  this  
  ( expression )  
  id-expression  
  lambda-expression  
  fold-expression  
  requires-expression  
+ do-expression
```

Add a new clause [\[expr.prim.do\]](#):

- 1 A *do-expression* provides a way to combine multiple statements into a single expression without introducing a new function scope.

```
do-expression:  
  do trailing-return-typeopt compound-statement
```

- 2 The *compound-statement* of a *do-expression* is a control-flow-limited statement ([\[stmt.label\]](#)).

Change 8.3 [\[stmt.expr\]](#) to disambiguate a *do* expression from a *do-while* loop:

- 1 Expression statements have the form

```
expression-statement:  
  expressionopt;
```

The expression is a *discarded-value* expression. All side effects from an expression statement are completed before the next statement is executed. An expression statement with the expression missing is called a *null statement*. The expression shall not be a *do-expression*.

[Note 1: Most statements are expression statements — usually assignments or function calls. A null statement is useful to supply a null body to an iteration statement such as a while statement ([\[stmt.while\]](#)). — end note]

Insert a disambiguation to 8.6.3 [\[stmt.do\]](#):

- 1 The expression is contextually converted to `bool`; if that conversion is ill-formed, the program is ill-formed.

- 1a The statement in the *do* statement shall not be a *return* statement.

- 2 In the *do* statement the substatement is executed repeatedly until the value of the expression becomes `false`. The test takes place after each execution of the statement.

Add to 8.7.1 [\[stmt.jump.general\]](#):

- 1 Jump statements unconditionally transfer control.

```
jump-statement:
    break ;
    continue ;
    return expr-or-braced-init-listopt ;
+ do return expr-or-braced-init-listopt ;
    coroutine-return-statement
    goto identifier ;
```

Add a new clause introducing a `do return` statement after 8.7.4 [\[stmt.return\]](#):

- 1 The `do` expression's value is produced by the `do return` statement.
- 2 A `do return` statement shall appear only within the *compound-statement* of a `do` expression.

§ 5 References

[P1371R3] Michael Park, Bruno Cardoso Lopes, Sergei Murzin, David Sankel, Dan Sarginson, Bjarne Stroustrup. 2020-09-15. Pattern Matching.

<https://wg21.link/p1371r3>

[P2561R1] Barry Revzin. 2022-10-11. An error propagation operator.

<https://wg21.link/p2561r1>

[P2688R0] Michael Park. 2022-10-16. Pattern Matching Discussion for Kona 2022.

<https://wg21.link/p2688r0>

[P2806R0] Barry Revzin, Bruno Cardoso Lopez, Zach Laine, Michael Park. 2023-02-14. `do` expressions.

<https://wg21.link/p2806r0>